

Métodos Numéricos I – Tarea 10

Pedro D. Llerenas
pedro.llerenas@cimat.mx

2025-11-09

1. Crear un código en C++ que resuelva el problema de mínimos cuadrados en el sentido de interpolación numérica para:

a) Un polinomio interpolador de $n + 1$ puntos de la forma $P(x) = a_0 + a_1x + \dots + a_nx^n$ para una función $f(x)$.

b) Una función interpoladora trigonométrica de $n + 1$ puntos para $f(x)$ de la forma

$$F_{\text{trig}}(x) = a_0 \cos\left(0 \frac{\pi x}{6}\right) + a_1 \cos\left(1 \frac{\pi x}{6}\right) + \dots + a_n \cos\left(n \frac{\pi x}{6}\right). \quad (1)$$

c) Una función interpoladora de base radial de $n + 1$ puntos para $f(x)$ de la forma

$$F_{\text{RBF}}(x) = a_0 e^{-r_0^2} + a_1 e^{-r_1^2} + \dots + a_n e^{-r_n^2}, \quad (2)$$

donde $r = (x - x_i)$ para $i = 0, \dots, n$.

Nota: Recordemos que el problema de mínimos cuadrados “regularizado” resuelve el sistema:

- Conjunto de entrenamiento

$$\mathcal{T} = \left\{ (x^{(k)}, y^{(k)}) \right\}_{k=1}^p, \quad y = f(x) = \sum_{i=1}^m \omega_i \varphi_i(x), \quad \Phi = (\varphi_1, \varphi_2, \dots, \varphi_n) \quad (3)$$

$$\begin{aligned} \Phi^T \Phi + \Lambda \omega^* &= \Phi^T y \\ \omega^* &= (\Phi^T \Phi + \Lambda)^{-1} \Phi^T y \end{aligned} \quad (4)$$

$$\omega^* = \begin{pmatrix} \omega_1^* \\ \omega_2^* \\ \vdots \\ \omega_m^* \end{pmatrix}, \quad \varphi_j = \begin{pmatrix} \phi_j(x^1) \\ \phi_j(x^2) \\ \vdots \\ \phi_j(x^p) \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}, \quad \Lambda = \begin{pmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_m \end{pmatrix}, \quad (5)$$

Encontrar los valores λ_i es todo un problema de optimización, para nuestros fines consideremos que los valores son constantes, es decir, $\lambda = \lambda_i, \forall i$

a) Para aproximar un conjunto de datos $\{(x_i, y_i)\}_{1 \leq i \leq m}$ con un polinomio algebraico

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 \quad (6)$$

de grado $n < m - 1$, por medio del procedimiento de mínimos cuadrados se deben seleccionar constantes $a_0, a_1, \dots, a_n$ para minimizar

$$E = \sum_{i=1}^m (y_i - P_n(x_i))^2. \quad (7)$$

Es decir, necesitamos $\frac{\partial E}{\partial a_j} = 0$. Esto se reduce a las $n + 1$ ecuaciones normales, que se escriben de la siguiente manera:

$$\begin{aligned}
 a_0 \sum_{i=1}^m x_i^0 + a_1 \sum_{i=1}^m x_i^1 + a_2 \sum_{i=1}^m x_i^2 + \cdots + a_n \sum_{i=1}^m x_i^n &= \sum_{i=1}^m y_i x_i^0, \\
 a_0 \sum_{i=1}^m x_i^1 + a_1 \sum_{i=1}^m x_i^2 + a_2 \sum_{i=1}^m x_i^3 + \cdots + a_n \sum_{i=1}^m x_i^{n+1} &= \sum_{i=1}^m y_i x_i^1, \\
 &\vdots \\
 a_0 \sum_{i=1}^m x_i^n + a_1 \sum_{i=1}^m x_i^{n+1} + a_2 \sum_{i=1}^m x_i^{n+3} + \cdots + a_n \sum_{i=1}^m x_i^{2n} &= \sum_{i=1}^m y_i x_i^n,
 \end{aligned} \tag{8}$$

Entonces, debemos de solucionar el sistema de ecuaciones lineal simétrico. Para esto, usamos Cholesky.

- b) De manera similar a la explicación anterior, con $x \mapsto \varphi_i(x)$, el problema se reduce a solucionar un sistema de ecuaciones, que de igual manera resolvemos con Cholesky.

2. Para evaluar los códigos del ejercicio 1, consideremos la función de Sutherland que define la viscosidad de un gas según la temperatura T en Kelvin (entre 1000), definida por

$$f(x) = m_0 \left(\frac{1000T}{T_0} \right)^{\frac{3}{2}} \left(\frac{T_0 + S_u}{1000T + S_u} \right). \tag{9}$$

Calculemos la viscosidad del oxígeno O_2 , para ello usemos $m_0 = 1.919e^{-2}$, $T_0 = 273$, y $S_u = 139$. Realiza lo siguiente:

- a) Evalúa los puntos $T_i = [0.273, 0.303, 0.323, 0.353, 0.423, 0.573, 1.473]$ en la función de Sutherland. Los puntos T_i corresponden a los $n + 1$ puntos x_i , y las evaluaciones son los valores y_i (estamos simulando que se ha realizado un experimento de laboratorio físico, es decir, sólo tenemos mediciones experimentales).
- b) Usando el valor de $\lambda = 0, 1e^{-5}, 1e^{-7}$, para $0 \leq i \leq n$, aplica las 3 funciones de interpolación programadas obtenidas del método de mínimos cuadrados en el inciso 1, con ellos calcula la viscosidad para $T = 1.2$, compara con el valor real obtenido directamente con la función de Sutherland.
- c) Grafica las soluciones obtenidas en el intervalo $[0.273, 1.5]$, usa incrementos de ~ 0.05 .
- d) Repite los incisos a), b) y c), pero con los nodos $T_i = [0.273, 0.473, 0.673, 0.873, 1.073, 1.273, 1.473]$. ¿Qué puedes concluir de los nodos y el comportamiento de las funciones de interpolación según λ ?

- a) Al evaluar los puntos T_i , obtenemos la siguiente tabla:

x_i	y_i
0.273	0.01919
0.303	0.0209156
0.323	0.0220237
0.353	0.0236279
0.423	0.0271333
0.573	0.033766
1.473	0.0614706

Table 1: Valores de la función Sutherland en los puntos dados.

- b) Para obtener los valores de cada interpolación, usamos

```
> make run-p2 ARGS="ex/2/init/Ti1.txt ex/2/init/lambda.txt"
```

Esto nos genera la tabla anterior, y las siguientes tablas, en el directorio ex/2/inter/.

T	λ	Valor	Error Absoluto
1.2	0	0.0535005	0.000914655
1.2	1e-05	0.0489938	0.00542131
1.2	1e-07	0.055693	0.00127788

Table 2: Valores de interpolación polinomial en $T = 1.2$ para distintos valores de λ .

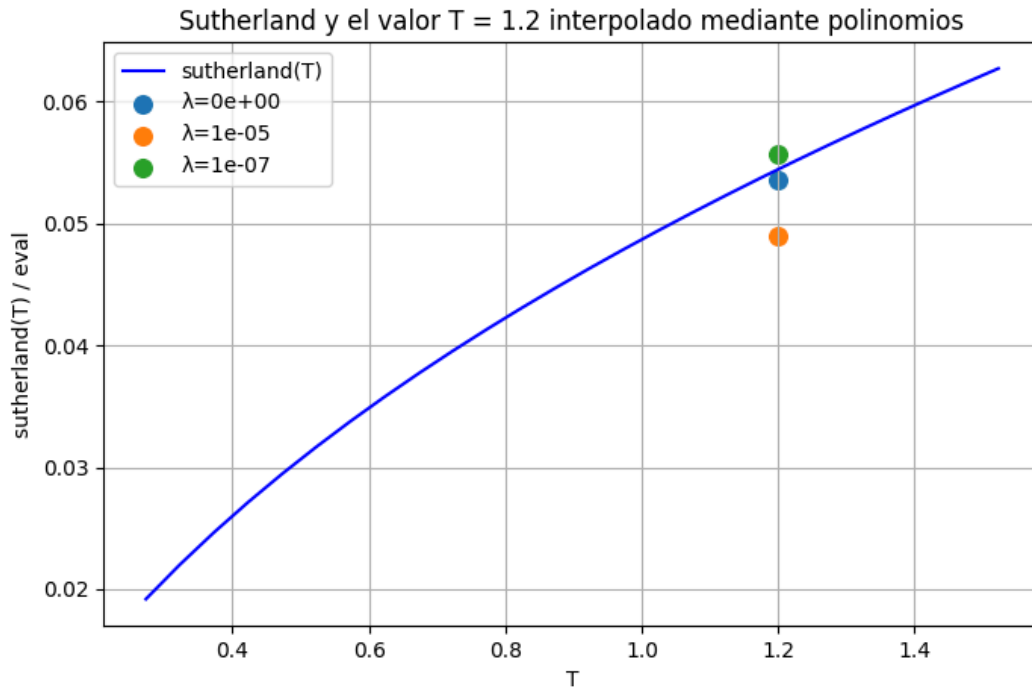
T	λ	Valor	Error Absoluto
1.2	0	0.284264	0.229848
1.2	1e-05	0.0593443	0.00492921
1.2	1e-07	0.0260663	0.0283488

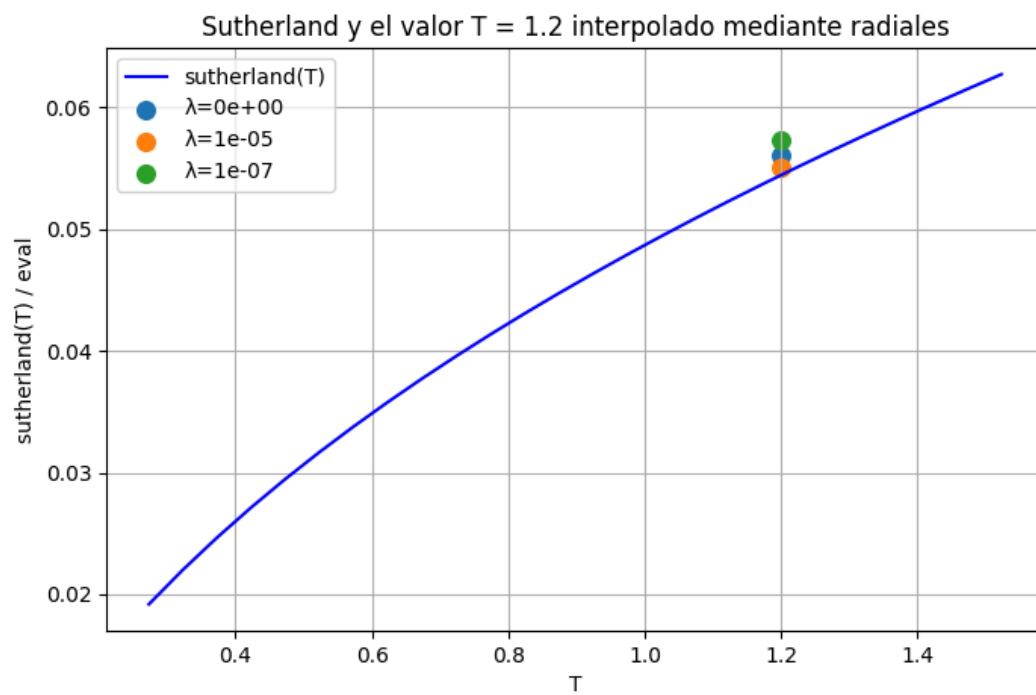
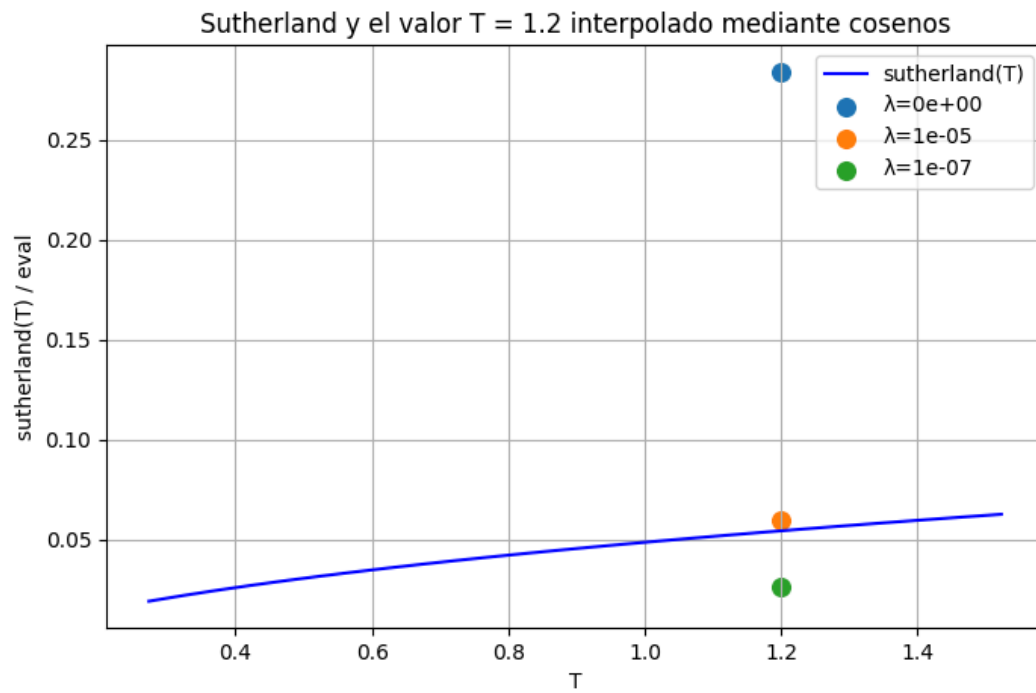
Table 3: Valores de interpolación con cosenos en $T = 1.2$ para distintos valores de λ .

T	λ	Valor	Error Absoluto
1.2	0	0.0560611	0.00164599
1.2	1e-05	0.0549939	0.000578769
1.2	1e-07	0.0573108	0.00289566

Table 4: Valores de interpolación con funciones radiales en $T = 1.2$ para distintos valores de λ .

c)





d)

T	$S(T)$
0.273	0.01919
0.473	0.0294624
0.673	0.0376873
0.873	0.0446755
1.073	0.0508306
1.273	0.0563814
1.473	0.0614706

Table 5: Valores de Sutherland en T_i

```
> make run-p2 ARGS="ex/2/init/Ti2.txt ex/2/init/lambda.txt"
```

Esto nos genera la tabla anterior, y las siguientes tablas, en el directorio ex/2/inter/.

T	λ	Valor	Error Absoluto
1.2	0	0.0544175	2.40576e-06
1.2	1e-05	0.0544015	1.36572e-05
1.2	1e-07	0.0544163	1.17073e-06

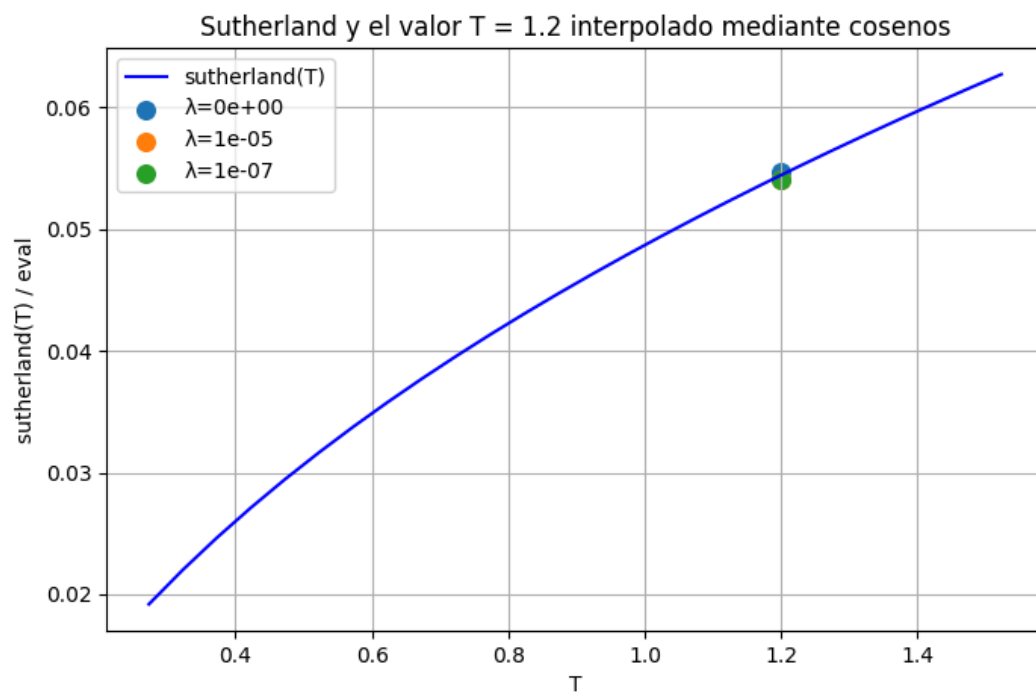
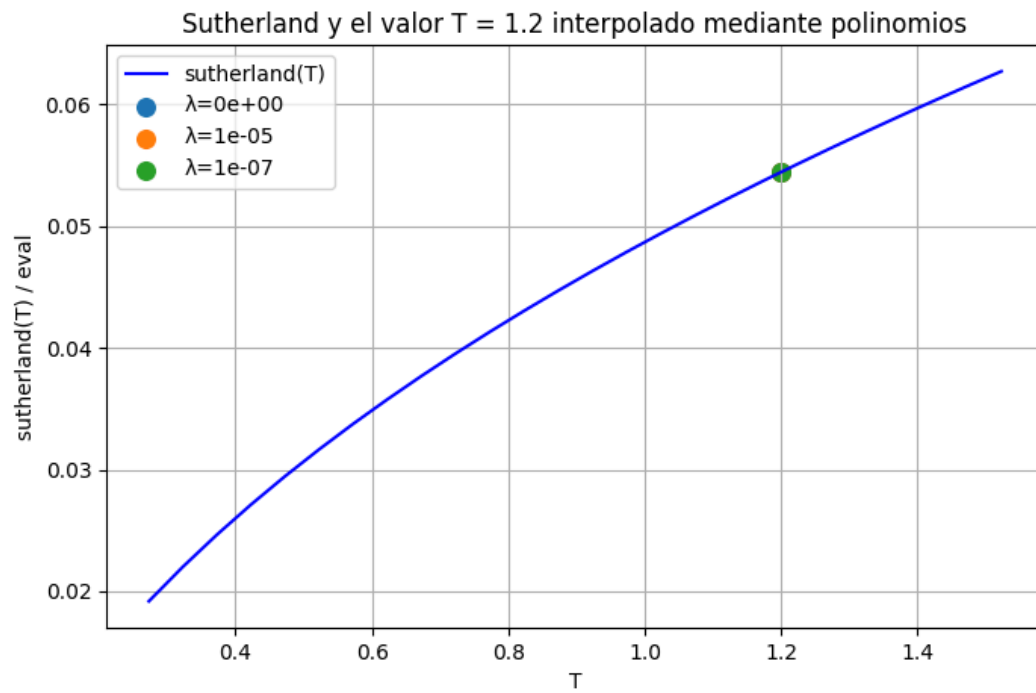
Table 6: Valores de interpolación polinomial en $T = 1.2$ para distintos valores de λ .

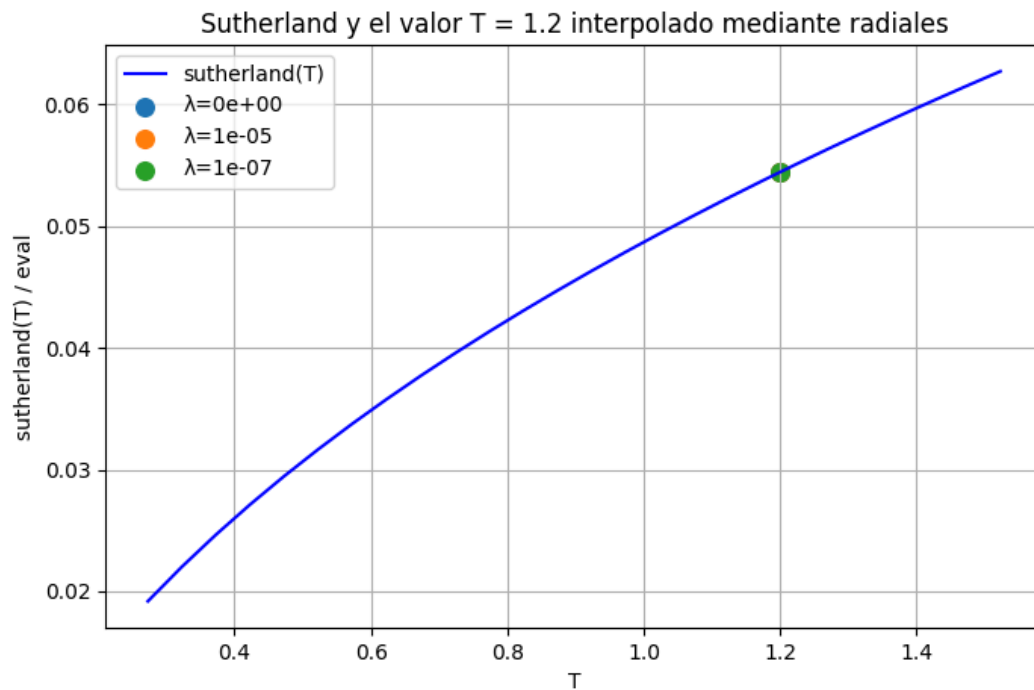
T	λ	Valor	Error Absoluto
1.2	0	0.0547209	0.000305836
1.2	1e-05	0.0540985	0.000316635
1.2	1e-07	0.0540558	0.000359354

Table 7: Valores de interpolación con cosenos en $T = 1.2$ para distintos valores de λ .

T	λ	Valor	Error Absoluto
1.2	0	0.0544114	3.71896e-06
1.2	1e-05	0.0544202	5.05207e-06
1.2	1e-07	0.0544137	1.37691e-06

Table 8: Valores de interpolación con funciones radiales en $T = 1.2$ para distintos valores de λ .





3. Crear una librería en C++ que calcule la aproximación a una integral en el intervalo $[a, b]$ arbitrario con los métodos de Newton-Cotes abierto ($n = 0, 1, 2, 3$), Newton-Cotes cerrado ($n = 1, 2, 3, 4$), y cuadratura gaussiana ($n = 1, 2, 3, 4, 5$). Con las librerías aproxima las siguientes integrales:

$$\int_0^{\pi/4} \sin(x) dx, \quad \int_1^{1.5} x^2 \ln x dx, \quad \int_0^1 x^2 e^{-x} dx, \quad (10)$$

Nota 1: Crea una tabla comparativa de los resultados. **Nota 2:** La primer integral es la vista en clase.

Las formulas de Newton-Cotes abiertas son aquellas que no toman los limites del intervalo en cuenta para la aproximación, mientras que las cerradas si. Esto puede ser una desventaja o ventaja, dependiendo de la función que es integrada. La cuadratura de Gauss utiliza los polinomios de Legendre como aproximaciones a la función integrada, donde primero se hace una transformación al intervalo $[-1, 1]$, que es donde los polinomios de Legendre son ortogonales. Se usa una tabla pre-cargada para hacer las estimaciones de las raíces y coeficientes correspondientes de cada n (ver `integration/gaussian.hpp`).

Para ejecutar los códigos, usamos

```
> make run-p3-a
> make run-p3-b
> make run-p3-c
```

Esto nos genera 9 tablas en los directorios `ex/sin`, `ex/xln` y `ex/xexp`, que presentamos a continuación:

n	$\int_0^{\frac{\pi}{4}} \sin(x)$
0	0.601118
1	0.0993292
2	0.0732147
3	0.0585738

Table 9: Aproximaciones por medio de Newton-Cotes abierto

n	$\int_0^{\frac{\pi}{4}} \sin(x)$
1	0.27768
2	0.292933
3	0.878732
4	0.292893

Table 10: Aproximaciones por medio de Newton-Cotes cerrado

n	$\int_0^{\frac{\pi}{4}} \sin(x)$
2	0.292867
3	0.292818
4	0.292893
5	0.292893

Table 11: Aproximaciones por medio de cuadratura gaussiana

n	$\int_1^{1.5} x^2 \ln(x)$
0	0.348662
1	0.0601042
2	0.0480679
3	0.0384536

Table 12: Aproximaciones por medio de Newton-Cotes abierto

n	$\int_1^{1.5} x^2 \ln(x)$
1	0.228074
2	0.192245
3	0.576759
4	0.192259

Table 13: Aproximaciones por medio de Newton-Cotes cerrado

n	$\int_1^{1.5} x^2 \ln(x)$
2	-0.0424486
3	0.415608
4	0.192259
5	0.192259

Table 14: Aproximaciones por medio de cuadratura gaussiana

n	$\int_0^1 x^2 \exp(-x)$
0	0.303265
1	0.0513
2	0.0397608
3	0.0319028

Table 15: Aproximaciones por medio de Newton-Cotes abierto

n	$\int_0^1 x^2 \exp(-x)$
1	0.18394
2	0.162402
3	0.48423
4	0.160611

Table 16: Aproximaciones por medio de Newton-Cotes cerrado

n	$\int_0^1 x^2 \exp(-x)$
2	0.15941
3	0.163076
4	0.160603
5	0.160603

Table 17: Aproximaciones por medio de cuadratura gaussiana

4. Investiga el método de extrapolación Richardson, y el método de Romberg, presenta un ejemplo. No olvides referenciar tu investigación.

Extrapolación de Richardson

La *extrapolación de Richardson*, atribuida a [Lewis Fry Richardson](#) (1881-1953) por sus artículos [1], [2], consiste en combinar diversas aproximaciones del valor α_0 para obtener una aproximación más precisa de α_0 .

Específicamente, sea $A(h)$ una función que aproxima el valor de α_0 para todo $h \neq 0$. Asumimos que para algún $k \geq 0$, podemos expandir la función $A(h)$ de la siguiente manera:

$$A(h) = \alpha_0 + \alpha_1 h + \cdots + \alpha_k h^k + R_{k+1}(h), \quad (11)$$

donde $|R_{k+1}(h)| \leq C_{k+1} h^{k+1}$. Las constantes C_{k+1} y los coeficientes α_i no dependen de h , por lo que

$$\alpha_0 = \lim_{h \rightarrow 0} A(h). \quad (12)$$

Para $\delta \in (0, 1)$, reescribimos (11) con $h \mapsto \delta h$:

$$A(\delta h) = \alpha_0 + \alpha_1 \delta h + \cdots + \alpha_k \delta^k h^k + R_{k+1}(\delta h). \quad (13)$$

Ahora, haciendo δ multiplicado por (11) menos (13), obtenemos

$$B(h) = \alpha_0 + \tilde{\alpha}_2 h^2 + \cdots + \tilde{\alpha}_k h^k + \tilde{R}_{k+1}(\delta h), \quad (14)$$

para $i = 2, \dots, k$, definimos $\tilde{\alpha}_i = \alpha_i \frac{\delta^i - \delta}{1 - \delta}$ y

$$\tilde{R}_{k+1}(h) = \frac{R_{k+1}(\delta h) - \delta R_{k+1}(h)}{1 - \delta}. \quad (15)$$

Con esta construcción, tenemos que $\tilde{\alpha}_i \neq 0 \Leftrightarrow \alpha_i$. En particular, si $\alpha_1 \neq 0$, entonces $A(h)$ es una aproximación de primer orden de α_0 , mientras que $B(h)$ es de orden al menos 2. En general, si $A(h)$ es una aproximación de α_0 de orden p , entonces el valor

$$B(h) = \frac{A(\delta h) - \delta^p A(h)}{1 - \delta^p} \quad (16)$$

aproxima a α_0 a un orden de al menos $p + 1$.

Por inducción, generamos el siguiente algoritmo de extrapolación de Richardson: Sea $n \geq 0$, $h > 0$ y $\delta \in (0, 1)$, construimos la secuencia

$$\begin{aligned} A_{m,0} &= A(\delta^m h), & m &= 0, \dots, n, \\ A_{m,q+1} &= \frac{A_{m,q} - \delta^{q+1} A_{m-1,q}}{1 - \delta^{q+1}}, & q &= 0, \dots, n-1, \\ & & m &= q+1, \dots, n, \end{aligned} \quad (17)$$

que se puede representar mediante el siguiente diagrama [3],

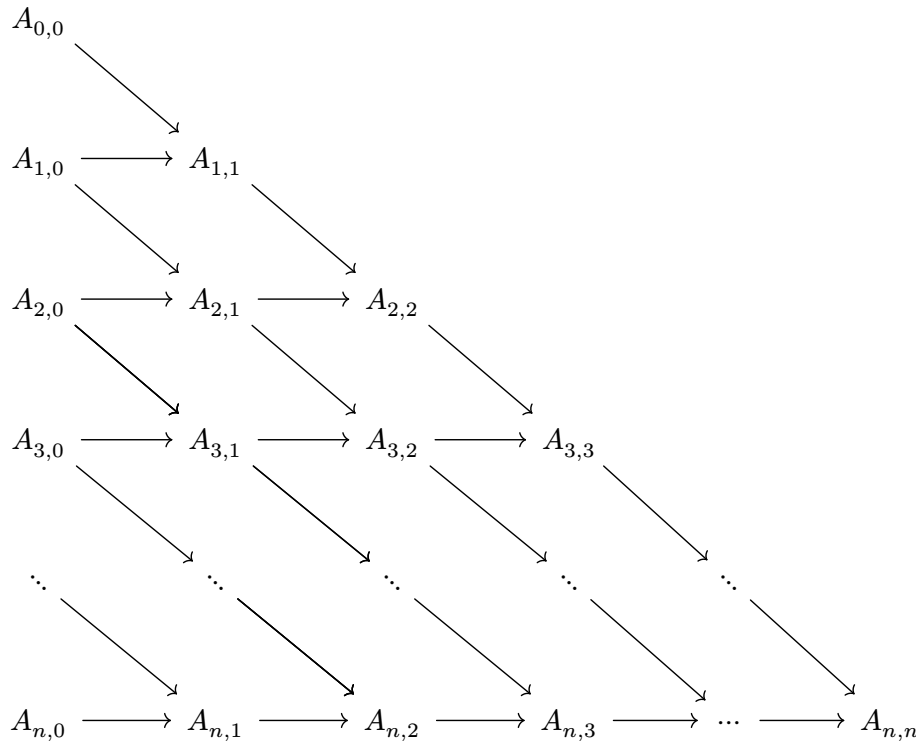


Figure 7: Diagrama de recurrencia. Las flechas indican el orden en el que se construyen los términos.

Se puede demostrar [3] que para $n \geq 0$ y $\delta \in (0, 1)$,

$$A_{m,n} = \alpha_0 + \mathcal{O}((\delta^m h)^{n+1}), \quad m = 0, \dots, n. \quad (18)$$

En particular, para $n = 0$, la convergencia a α_0 es de $\mathcal{O}(\delta^m h)$.

Ejemplo 1.

Podemos emplear extrapolación de Richardson para aproximar la derivada de $f(x) = xe^{-x} \cos(2x)$ en el punto $x = 0$. De acuerdo al proceso explicado, necesitamos una función que aproxime este valor. Proponemos

$$A(h) = \frac{f(x+h) - f(x)}{h}, \quad (19)$$

una elección obvia al ser la derivada la que intentamos aproximar. Tomemos $\delta = 0.5$, $n = 5$ y $h = 0.1$. La siguiente tabla nos presenta los errores absolutos $E_{m,k}$ del valor real α_0 y la aproximación $A_{m,k}$.

$E_{m,0}$	$E_{m,1}$	$E_{m,2}$	$E_{m,3}$	$E_{m,4}$	$E_{m,5}$
0.113	—	—	—	—	—
5.3×10^{-2}	6.1×10^{-3}	—	—	—	—
2.6×10^{-2}	1.7×10^{-3}	2.2×10^{-4}	—	—	—
1.3×10^{-2}	4.5×10^{-4}	2.8×10^{-5}	5.5×10^{-7}	—	—
6.3×10^{-2}	1.1×10^{-4}	3.5×10^{-6}	3.1×10^{-8}	3.0×10^{-9}	—
3.1×10^{-3}	2.9×10^{-5}	4.5×10^{-7}	1.9×10^{-9}	9.9×10^{-11}	4.9×10^{-12}

Metodo de Romberg

Se le atribuye a [Werner Romberg](#) por su artículo [4] describiendo el método de integración. Consiste en utilizar la extrapolación de Richardson para aproximar la integral de una función $f : \mathbb{R} \rightarrow \mathbb{R}$ mediante el método del trapecio. Es decir, si nuestra función de aproximación es el método del trapecio para aproximar la integral, entonces podemos extrapolar una mejor aproximación.

Teorema 0.1. Sea $f \in C^{2k+2}([a, b])$ para $k \geq 0$. La aproximación de $\alpha_0 = \int_a^b f(x) dx$ dada por la regla del trapecio es

$$I_{1,m}(f) = \alpha_0 + \sum_{i=1}^k \frac{B_{2i}}{(2i)!} h_m^{2i} (f^{(2i-1)}(b) - f^{(2i-1)}(a)) + \frac{B_{2k+2}}{(2k+2)!} h_m^{2k+2} (b-a) f^{(2k+2)}(\eta), \quad (20)$$

donde $h_m = \frac{b-a}{m}$ para $m \geq 1$, $\eta \in (a, b)$ y

$$B_{2j} = (-1)^{j-1} \left[\sum_{n=1}^{\infty} \frac{2}{(2n\pi)^{2j}} \right] (2j)!, \quad \text{para } j \geq 1. \quad (21)$$

Podemos notar que [Teorema 0.1](#) es un caso especial de [\(11\)](#), usando $h = h_m^2$ y $A(h) = I_{1,m}(f)$. Entonces, el algoritmo de Richardson nos provee la siguiente recursión:

$$\begin{aligned} A_{m,0} &= A(\delta^m h), & m &= 0, \dots, n, \\ A_{m,q+1} &= \frac{A_{m,q} - \delta^{2(q+1)} A_{m-1,q}}{1 - \delta^{2(q+1)}}, & q &= 0, \dots, n-1, \\ & & m &= q+1, \dots, n, \end{aligned} \quad (22)$$

Usando $h = b-a$ y $\delta = 0.5$ en [\(22\)](#), con $s = 2^m$ sub-intervalos de longitud $h_s = \frac{b-a}{2^m}$, tenemos

$$\begin{aligned} A_{m,0} &= I_{1,s} \left(\frac{b-a}{2^m} \right), & m &= 0, \dots, n, \\ A_{m,q+1} &= \frac{4^{q+1} A_{m,q} - A_{m-1,q}}{4^{q+1} - 1}, & q &= 0, \dots, n-1, \\ & & m &= q+1, \dots, n. \end{aligned} \quad (23)$$

Entonces, tenemos la siguiente convergencia:

$$A_{m,n} = \int_a^b f(x) dx + \mathcal{O}(h_s^{2(n+1)}), \quad n \geq 0. \quad (24)$$

Ejemplo 2.

Aproximamos el valor de la integral $\alpha_0 = \int_0^\pi e^x \cos(x) dx = -\frac{e^\pi + 1}{2}$. Obtenemos la siguiente tabla

k	E_k
0	22.71
1	0.4775
2	5.926×10^{-2}
3	7.410×10^{-5}
4	8.923×10^{-7}
5	6.850×10^{-11}
6	5.330×10^{-14}
7	0

Bibliography

- [1] L. F. Richardson, “IX. The approximate arithmetical solution by finite differences of physical problems involving differential equations, with an application to the stresses in a masonry dam,” *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, vol. 210, no. 459–470, pp. 307–357, 1911, doi: [10.1098/rsta.1911.0009](https://doi.org/10.1098/rsta.1911.0009).
- [2] L. F. Richardson and J. A. Gaunt, “VIII. The deferred approach to the limit,” *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, vol. 226, no. 636–646, pp. 299–361, 1927, doi: [10.1098/rsta.1927.0008](https://doi.org/10.1098/rsta.1927.0008).
- [3] A. Quarteroni, R. Sacco, and F. Saleri, “Richardson Extrapolation,” in *Numerical Mathematics*, Springer, 2000, pp. 387–389.
- [4] W. Romberg, “Vereinfachte numerische Integration,” *Mathematische Nachrichten*, vol. 1, no. 1, pp. 263–278, 1955.